

#UNIT-2

Quick Work

Page No.:

Date:

M T W T F S S

Regression -

- You need to find a function that maps some features or variables to others sufficiently well.
- Dependent features are called dependent variables, outputs or responses
- Independent features are called independent variables, inputs or predictors
- Regression problems usually have one continuous and unbounded dependent variable.
- Inputs can be continuous discrete or even categorical
- It is a common practice to denote outputs with y and inputs with x . If there are 2 or more independent variables they can be represented as the vector $x = (x_1, \dots, x_t)$ where $t = \text{no. of inputs}$

Linear Regression -

- Linear model where the model assumes a linear relationship between input variable (x) & single output variable (y).

- Types:

- (i) Simple Linear Regression
- (ii) Multiple Linear Regression

$$y = a + bx + e$$

where $a = \text{intercept}$

$b = \text{slope}$

$e = \text{residual}$

- Advantages:

- (i) Simple & easy
- (ii) Fast
- (iii) Interpretability
- (iv) Works well with small data

- Disadvantages:

- (i) Linearity assumption
- (ii) Sensitive to outliers
- (iii) Multi collinearity issues
- (iv) Normality assumption
- (v) Poor for complex patterns

- When implementing linear regression of some dependent variable y on set of independent variables $x = (x_1, \dots, x_f)$, where $f = \text{no. of predictors}$ you assume a linear relationship between y and x :

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_f x_f + \epsilon \quad [\text{Regression eqn}]$$

where, $\beta_0, \beta_1, \dots = \text{regression coefficients}$
 $\epsilon = \text{random error}$

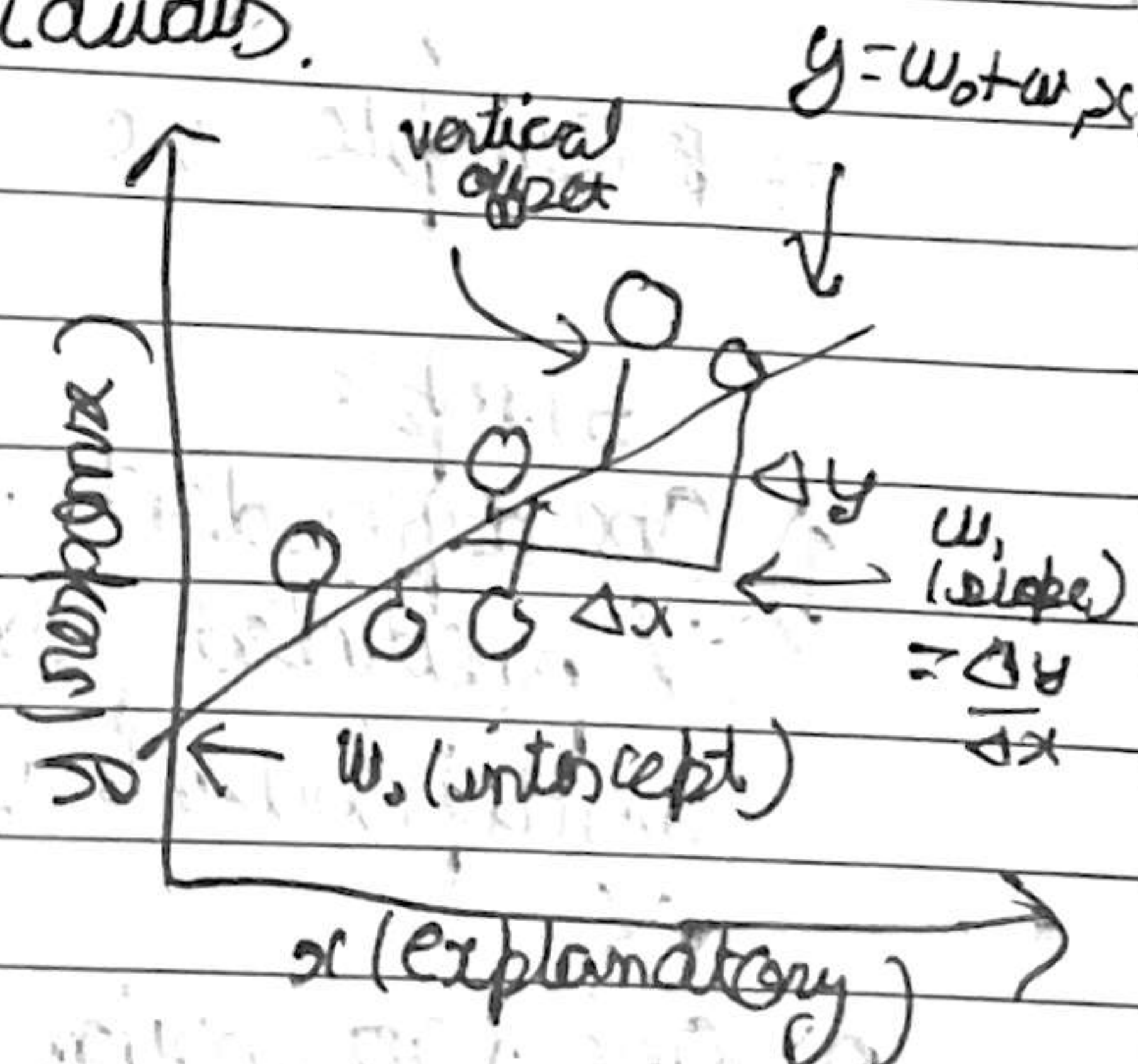
- It calculates the estimators of the regression coefficients or simply the predicted weights, denoted by $b_0, b_1, \dots, b_f$. They define the estimated regression function

$$f(x) = b_0 + b_1 x + \dots + b_f x_f$$

Captures the dependencies between inputs & outputs sufficiently well.

- The estimated or predicted response $f(x_i)$ for each observation $i = 1, \dots, n$ should be as close as possible to the corresponding actual response y_i .

The differences $y_i - f(x_i)$ for all observations $i = 1, \dots, n$ are called the residuals.



Multi linear regression -

Extension of single linear regression.

It models relationship between $X_1, X_2, \dots, X_n$ and Y

Relationship can be expressed as:

$$y = \alpha_0 + \alpha_1 X_1 + \alpha_2 X_2 + \dots + \alpha_n X_n$$

X_1	Size of (canal)	No. of BR	Price (Cr)
	X_1	X_2	Y
1		3	2
2		4	5
3		7	9

$$y = \alpha_0 + \alpha_1 X_1 + \alpha_2 X_2$$

$$\hat{b} = ((X^T X)^{-1} X^T) y$$

$$b^0 = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \end{bmatrix}$$

$$X = \begin{bmatrix} 1 & 1 & 3 \\ 1 & 2 & 4 \\ 1 & 3 & 7 \end{bmatrix}$$

$$X^T = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 3 & 4 & 7 \end{bmatrix}$$

Simple vs Multiple Regression:

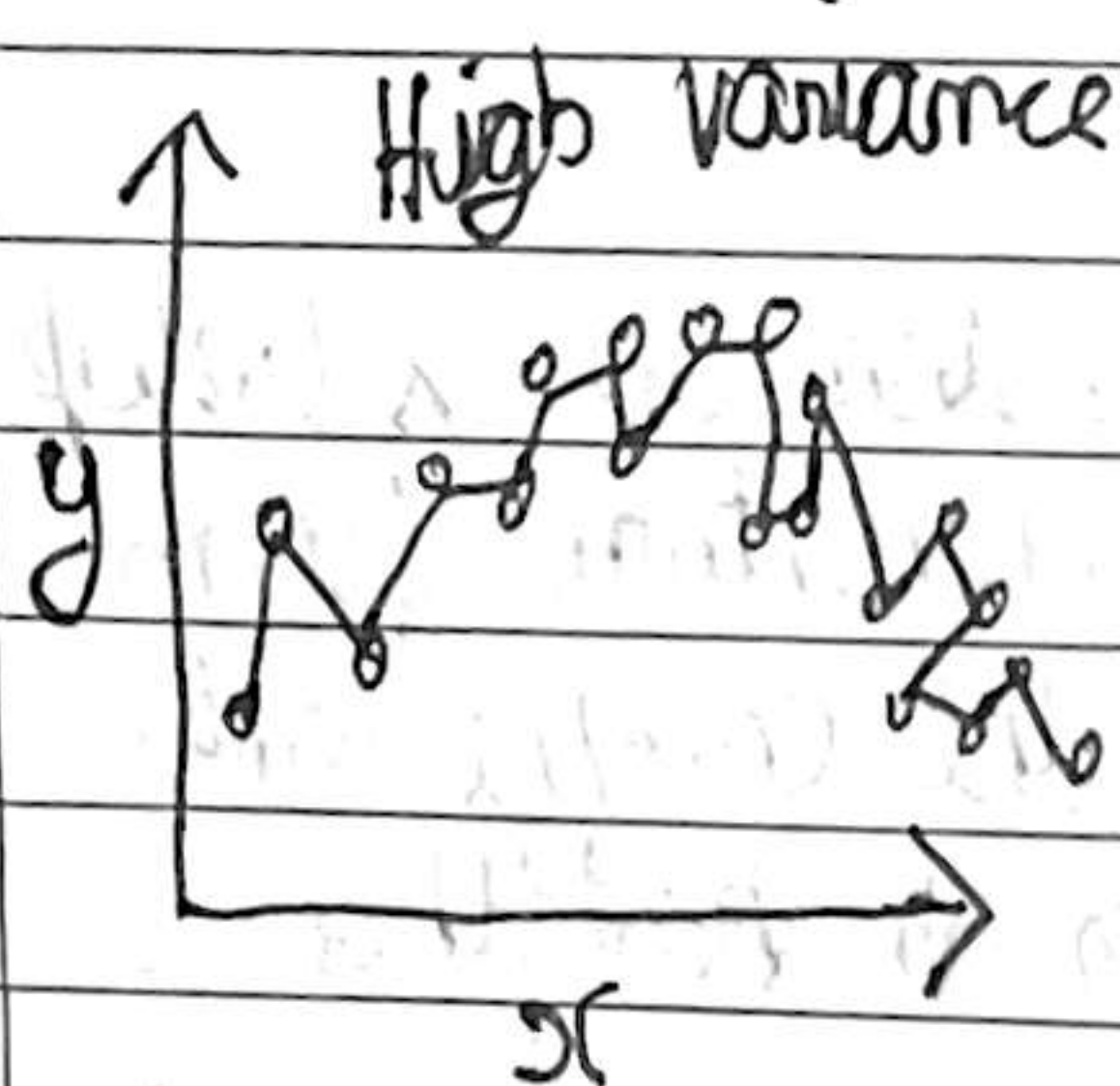
Simple	Multiple
1) One dependent variable Y predicted from one independent variable X	1) One dependent variable Y predicted from a set of independent variables $(X_1, X_2, \dots, X_k)$
2) One regression coefficient	2) One regression coefficient for each independent variable
3) r^2 : proportion of variation in dependent variable Y predictable from X	3) R^2 : proportion of variation in dependent variable Y predictable by set of independent variables (X 's)

Bias -

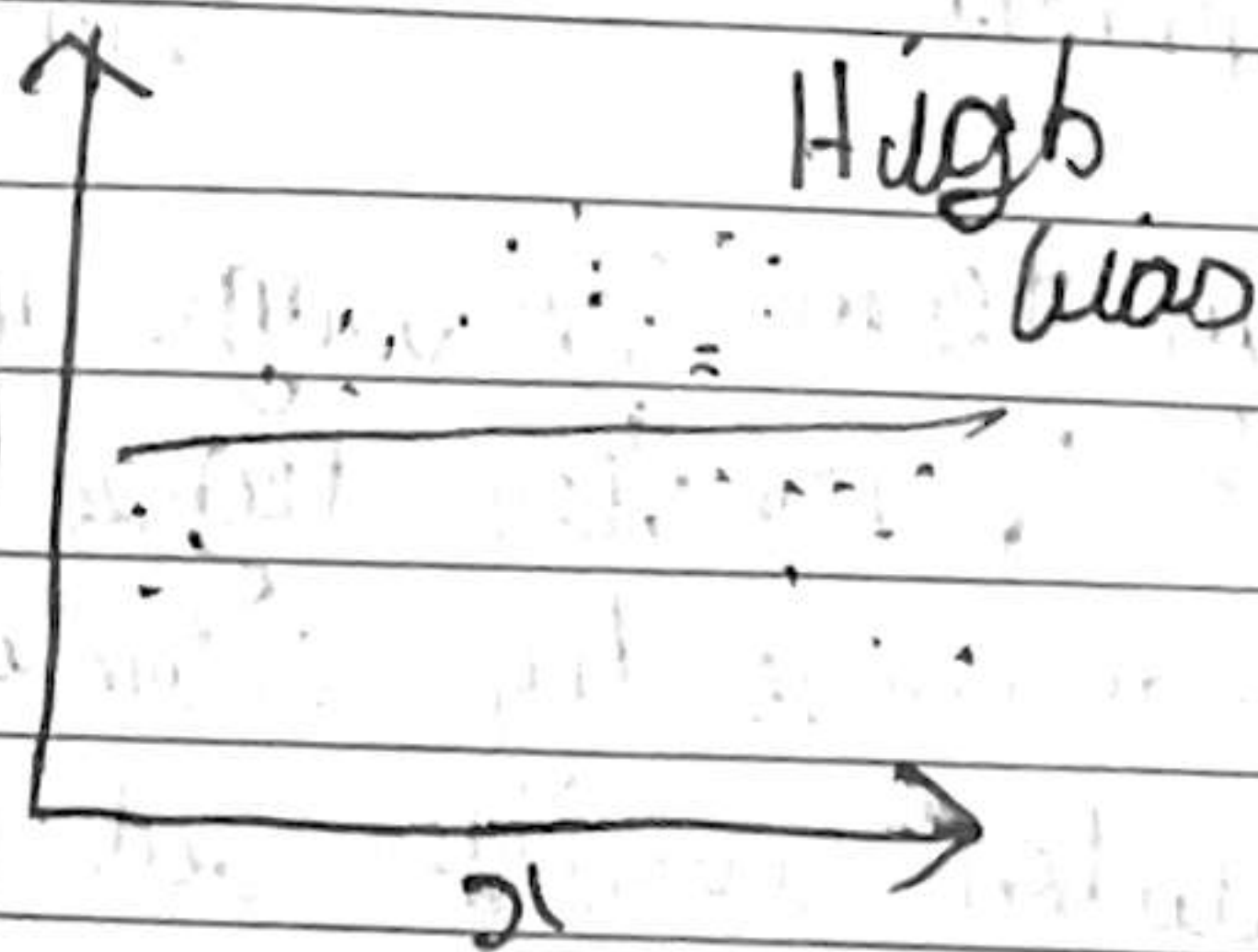
- Underlying assumptions that are made by data to simplify the target function
- Help us generalize the data better & make the model less sensitive to single data points
- Examples of high bias algorithms include linear regression, logistic regression etc.
- This leads to the underfitting of the model sometimes

Variance -

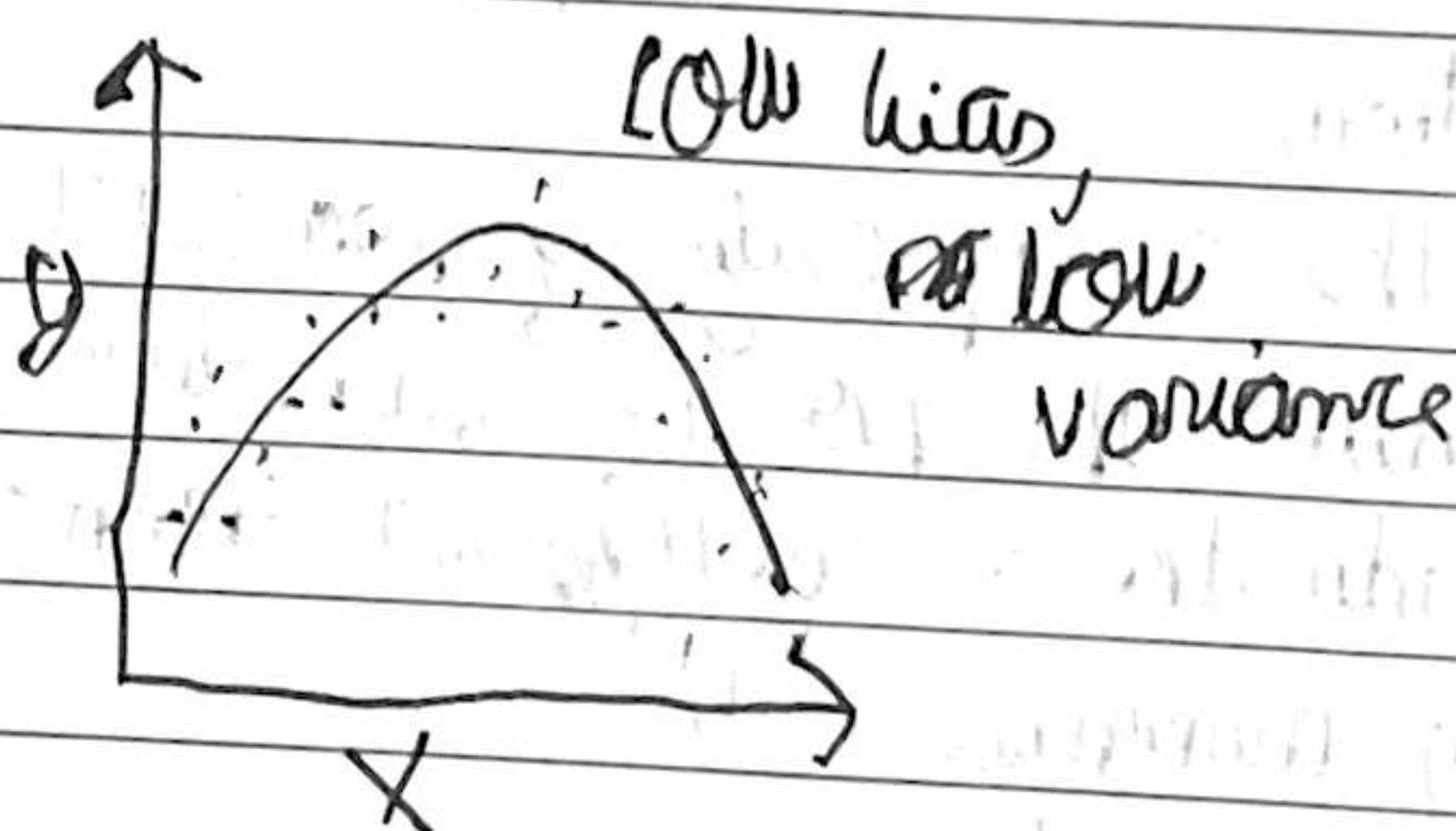
- Type of error that occurs due to a model's sensitivity to small fluctuations in the dataset
- High variance would cause an algorithm to model the outliers/noise in the training set, commonly referred as overfitting
- eg- Decision tree, KNN, etc



overfitting



Underfitting



Good balance

Regularized Regression-

- Used to avoid overfitting of the data especially when the trained & tested data are much varying.
- Implemented by adding a penalty term to the best fit derived from the trained data, in order to achieve a lesser variance with the tested data & also restricts the influence of predictor variables over the ~~input~~ output variable by compressing their coefficients.

Ridge Regression-

- Implemented by adding bias to a multilinear regression model to expect a much accurate regression with the tested data at a cost of losing accuracy for the trained data.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_K x_K$$

↑ output variable
 ↑ predictor variables

- Penalty term for ridge regression is $\lambda (\text{slope})^2$, where λ denotes degree of deflection from original curve by restricting the coefficients of predictor variables but never makes them zero.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_K x_K + \lambda (\text{slope})^2$$

- Limitation:

- (eg) ↓ the complexity of a model but does not reduce the no. of variables since it never leads to a coefficient ~~becomes~~ zero rather only minimizes it.

Lasso Regularization -

- Similar to Ridge but only differs in the penalty term.

- Penalty for lasso regression is $\lambda |\text{slope}|$.

- Can even eliminate the variables by making their coefficients to zero thus removing the variables that have high covariance with other predictor variables.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \lambda |\text{slope}|$$

- Limitation:

(i) Can select at most n predictors when predictors exceed observations, possibly relevant variables.

(ii) In the presence of highly collinear variables, it randomly selects one, making their interpretation unreliable.

Bayesian Linear Regression -

- Uses Baye's theorem to calculate conditional probabilities, aiding precise predictions in machine learning.

- Naive Bayes, a simplified version, reduces computation time & cost.

- Helps estimate the probability of an event given that another event has occurred, linking conditional and marginal probabilities.

- In the Bayesian viewpoint, we formulate linear regression using probability distributions rather than point estimates.

$$P(\beta|y, x) = \frac{P(y|\beta, x) * P(\beta|x)}{P(y|x)}$$

Posterior
Probability
distribution of the model
parameters

Posterior = Likelihood * Prior
Normalization

Logistic Regression -

- Used when dependent variable (target) is categorical.

- Types:

(i) Binomial = target variable can have only 2 possible types: "0" or "1"

(ii) Multinomial = target variable can have 3 or more possible types which are not ordered

(iii) Ordinal = target variables with ordered categories

- Primarily used for binary classification where target is categorical.

- Logistic function (sigmoid function) is an S-shaped curve that takes a real valued number & maps it to a value between 0 and 1 $\rightarrow$ probabilities

$$y = \frac{1}{1 + e^{-z}}$$

Linear equation: $y = b_0 + b_1 x$

Apply Sigmoid function

$$P(x) = \text{sigmoid}(y)$$

$$P(x) = \frac{1}{1 + e^{-y}}$$

$$P(x) = \frac{1}{1 + e^{-(b_0 + b_1 x)}}$$

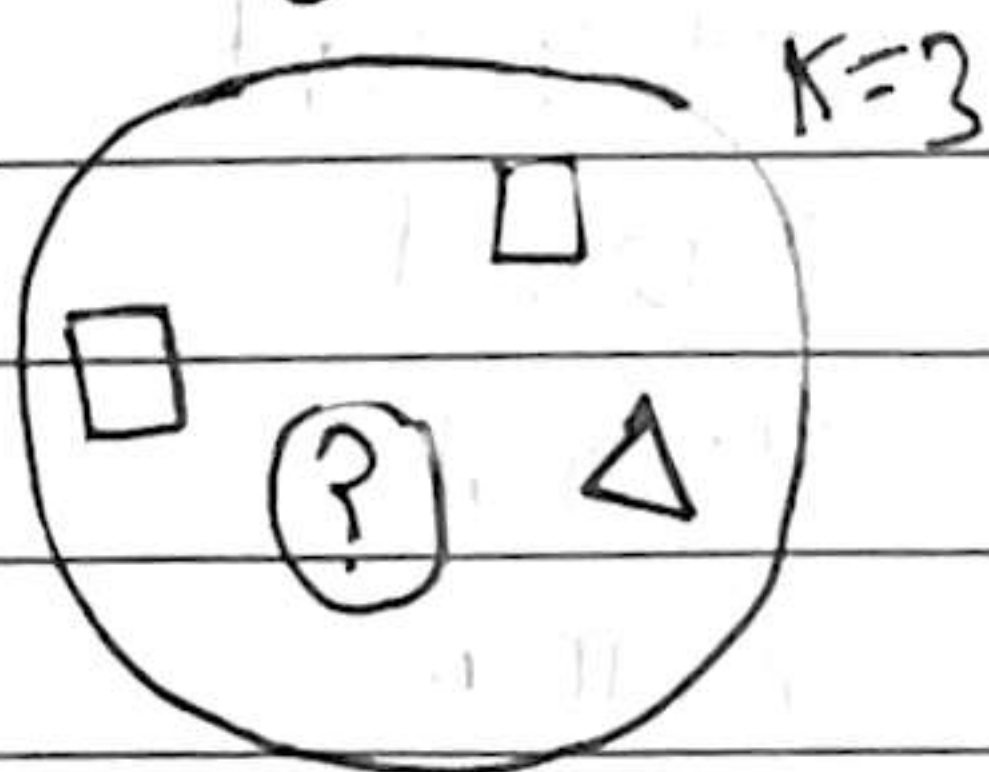
Linear vs Logistic Regression -

Logistic

Aspect	Linear	Logistic
1) Def	Predict a continuous dependent variable	Predict a categorical dependent variable
2) Estimation method	Least square estimation	Maximum likelihood estimation
3) Best fit line	Straight line	Curve
4) Relationship	Linear relationship between IV & DV	Linear not mandatory
5) Output	Predicted integer value	Predicted binary value

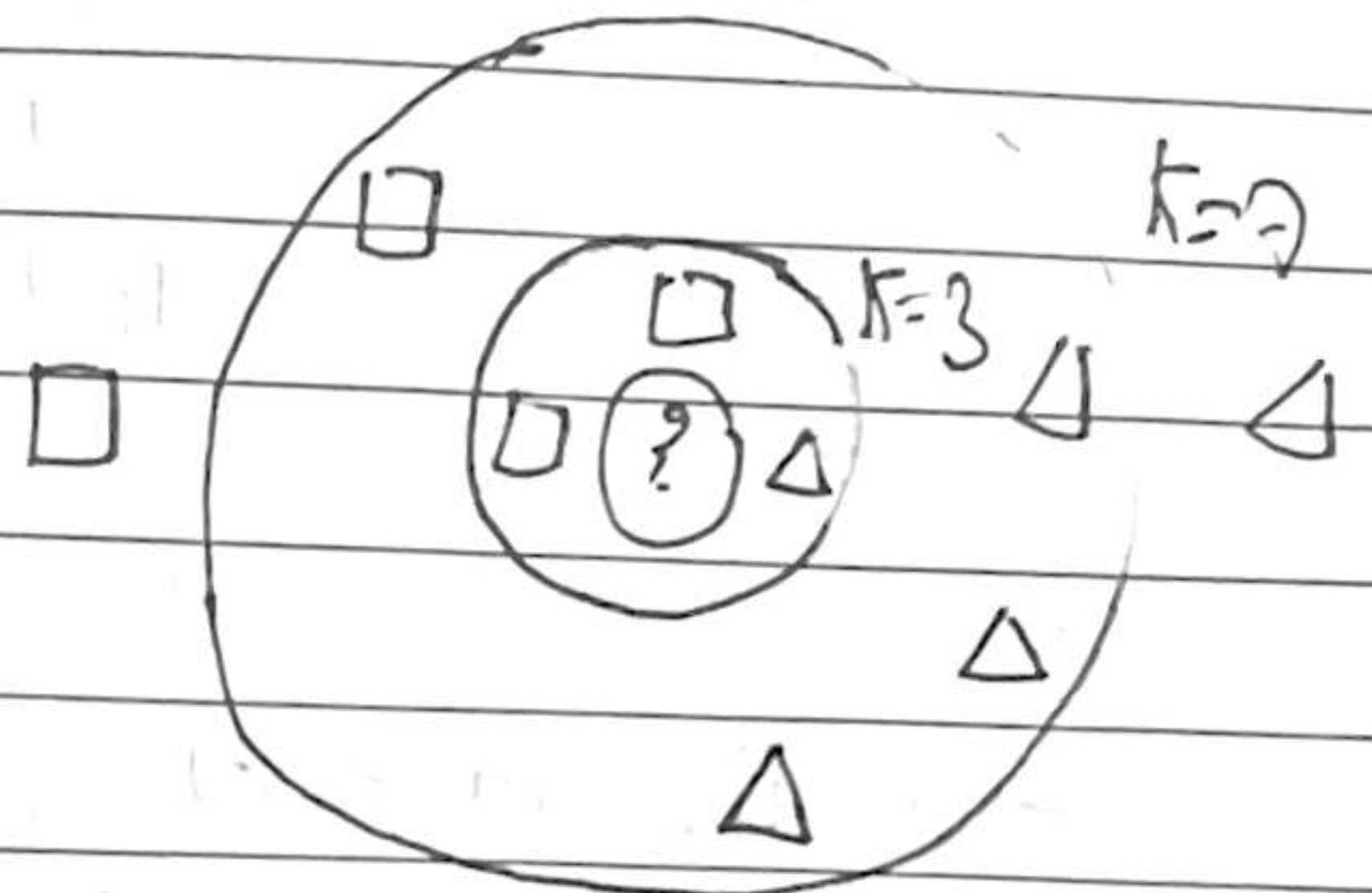
KNN Algorithm -

- Classifies a data point based on how its neighbors are classified.
- Stores all available cases & classifies new ~~on~~ cases based on a similarity measure
- K refers to the number of nearest neighbors to include in the majority voting process.
- Based on feature similarity
- Choosing right value of K is a process called parameter tuning & is important for better accuracy.



At $K=3$,
we classify?

as □



But at
 $K=7$,
we classify?
as △

- To choose a value of " K ":

- $K = \sqrt{n}$, where n = total no. of data points
- An odd value of K is often selected to avoid ties between classes.
- A higher value of K reduces the effect of noise but may increase bias

- We use KNN when:

- Dataset is small
- Data is labeled
- Data is noise free

Working of KNN algorithm-

- Consider a dataset having 2 variables: height (cm) and weight (kg) and each point is classified as Normal or underweight.

Weight (x2)	Height (y2)	Class
51	167	Underweight
62	182	Normal
69	176	Normal
64	173	Normal
65	172	Normal
56	174	Underweight
58	164	Normal
57	173	Normal
55	170	Normal

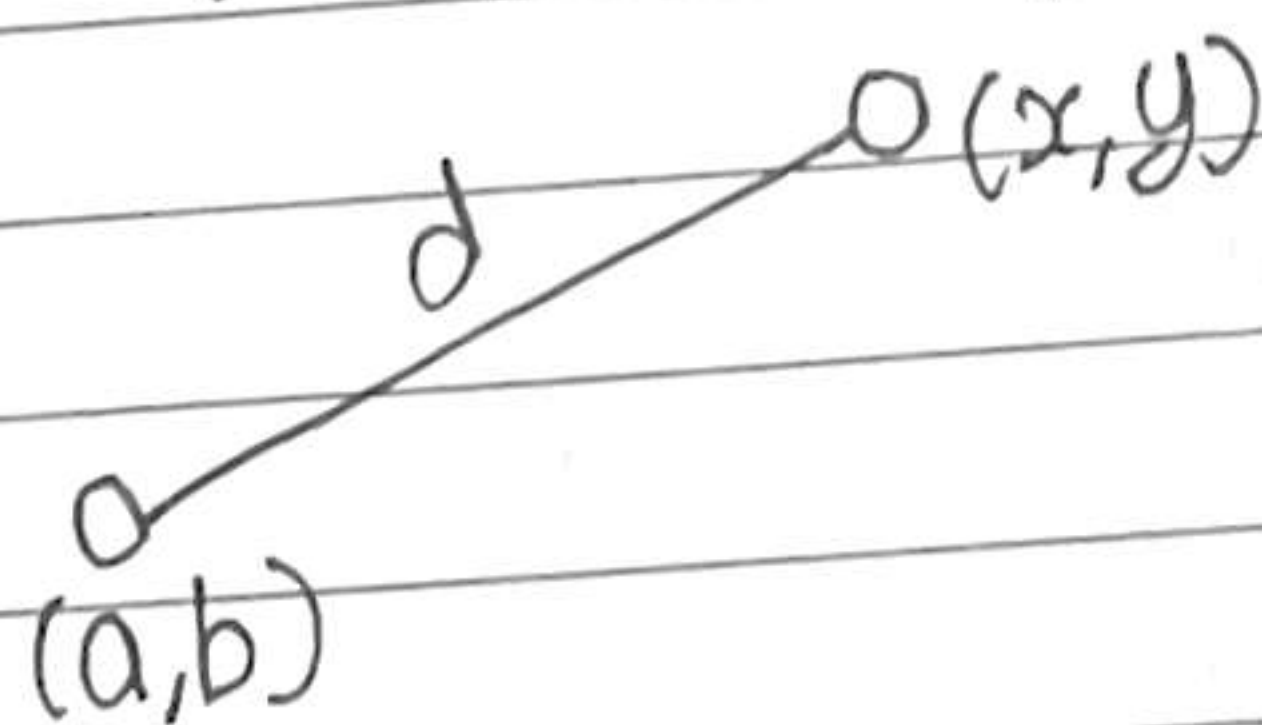
- On the basis of the given data, we have to classify the below set as Normal or Underweight using KNN?

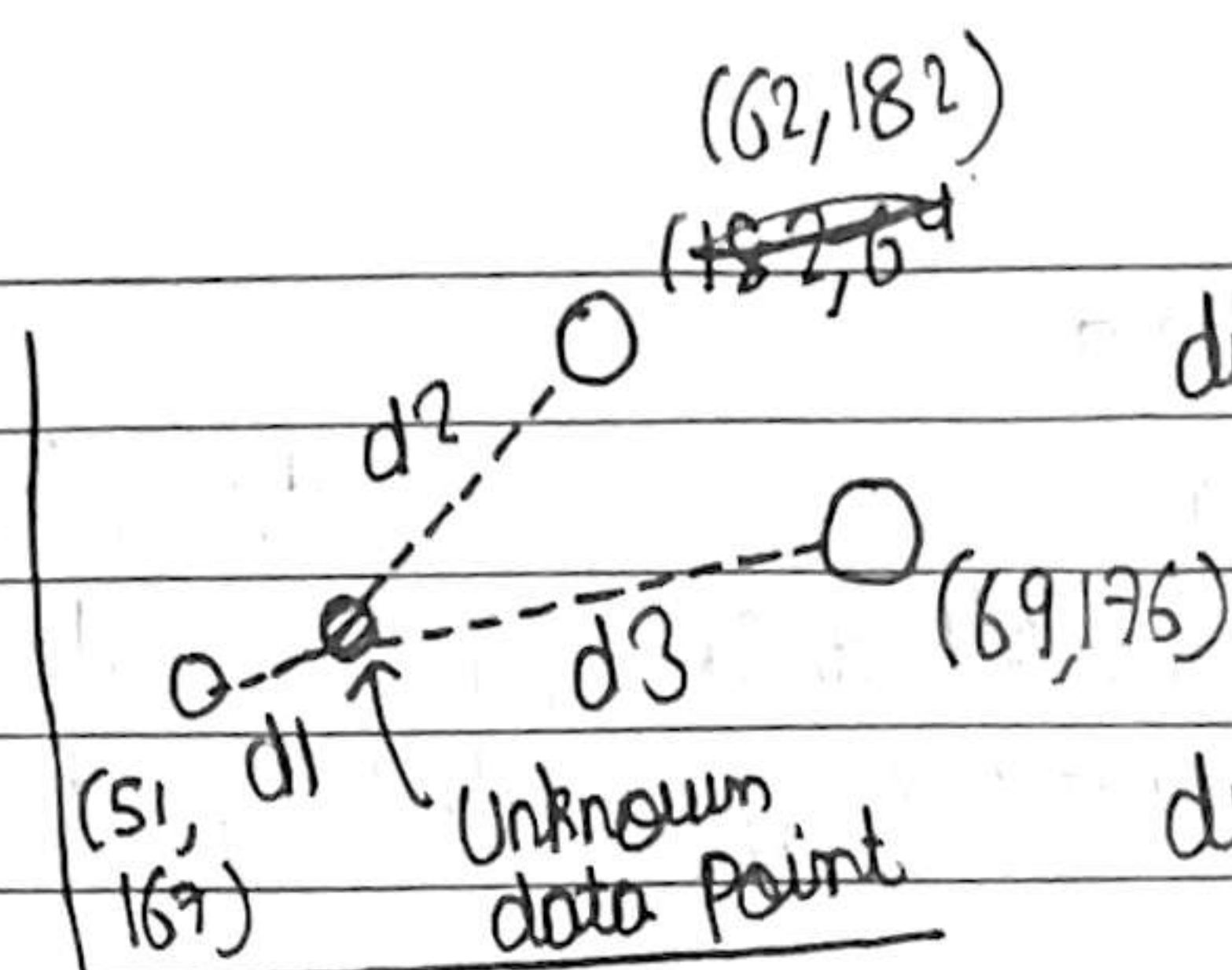
57 170

- To find the nearest neighbors, we will calculate Euclidean distance

- According to the Euclidean distance formula, the distance between 2 points in the plane with coordinates (x,y) and (a,b) is given by:

$$\text{dist}(d) = \sqrt{(x-a)^2 + (y-b)^2}$$





$$\text{dist}(d_1) = \sqrt{(170-167)^2 + (57-51)^2} \\ = \sim 6.7$$

$$\text{dist}(d_2) = \sqrt{(170-182)^2 + (57-62)^2} \\ = \sim 13$$

Similarly, we will

$$\text{dist}(d_3) = \sqrt{(170-176)^2 + (57-69)^2} \\ = \sim 13.4$$

Calculate Euclidean distance

of unknown data point from all the points in the dataset.

Hence, we calculated the Euclidean distance of unknown data points from all the points as shown:

	Weight (x2)	Height (y2)	Class	Euclidean Distance
	51	167	Underweight	6.7
	62	182	Normal	13
where (x1, y1) = (57, 170)	69	176	Normal	13.4
	64	173	Normal	7.6
whose class we have to classify	65	172	Normal	8.2
	56	174	Underweight	4.1
	58	169	Normal	1.4
	57	173	Normal	3
	55	170	Normal	2

$K=3$

~~n=10~~ $n=10$

~~avg~~ $\text{avg}(10) = 3.1$

* Calculate the nearest neighbor at $K=3$

=> select only the 3 smallest distances

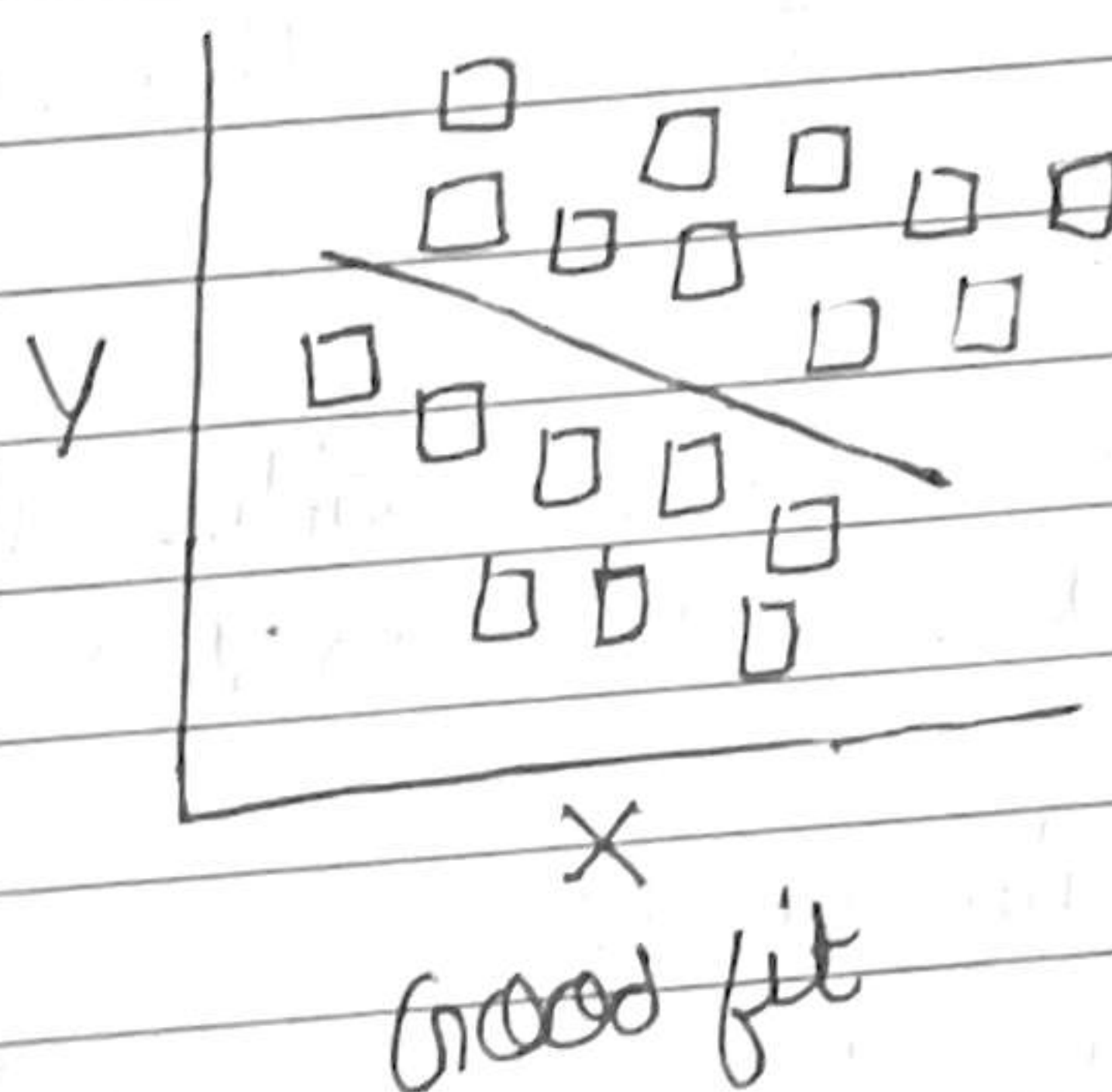
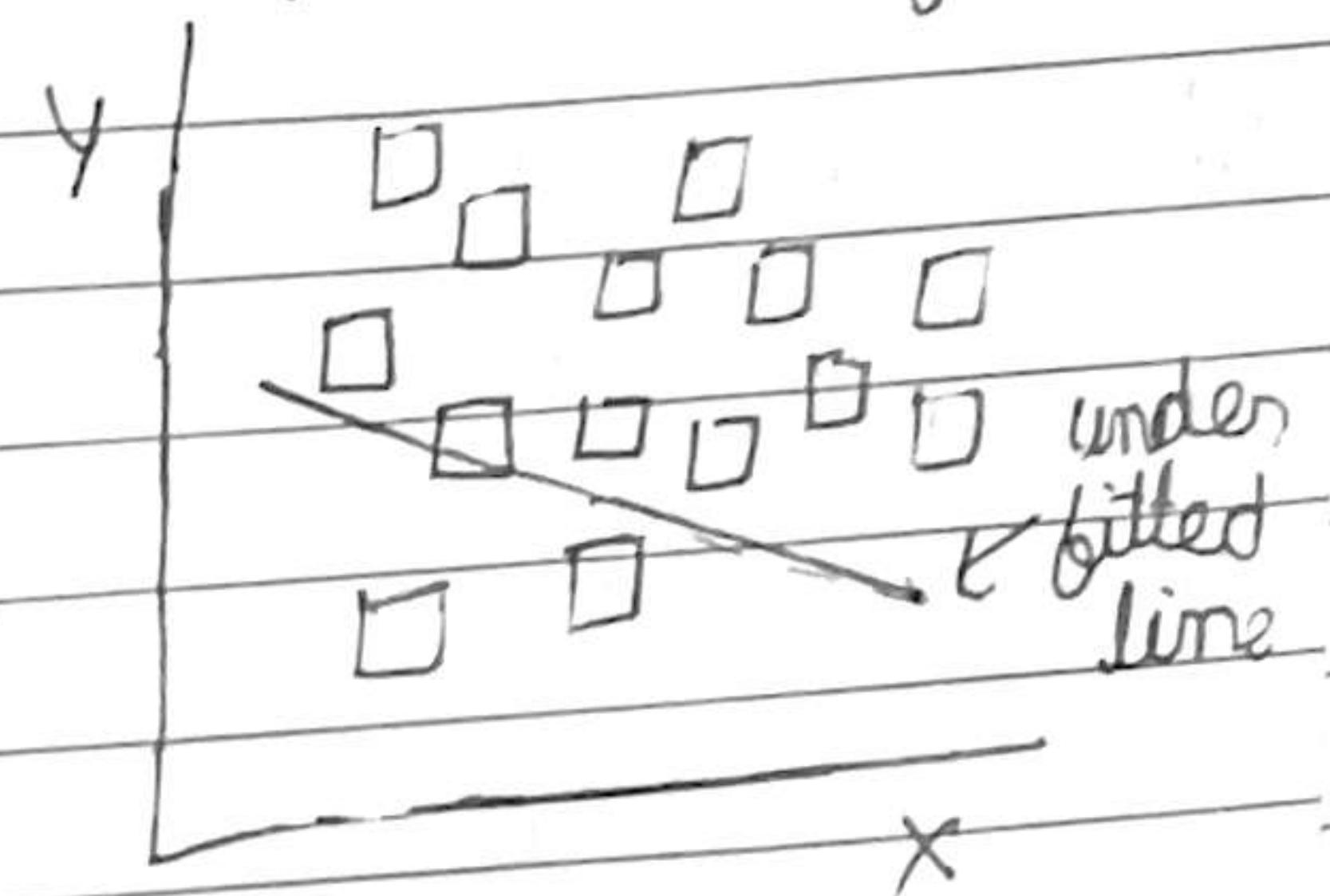
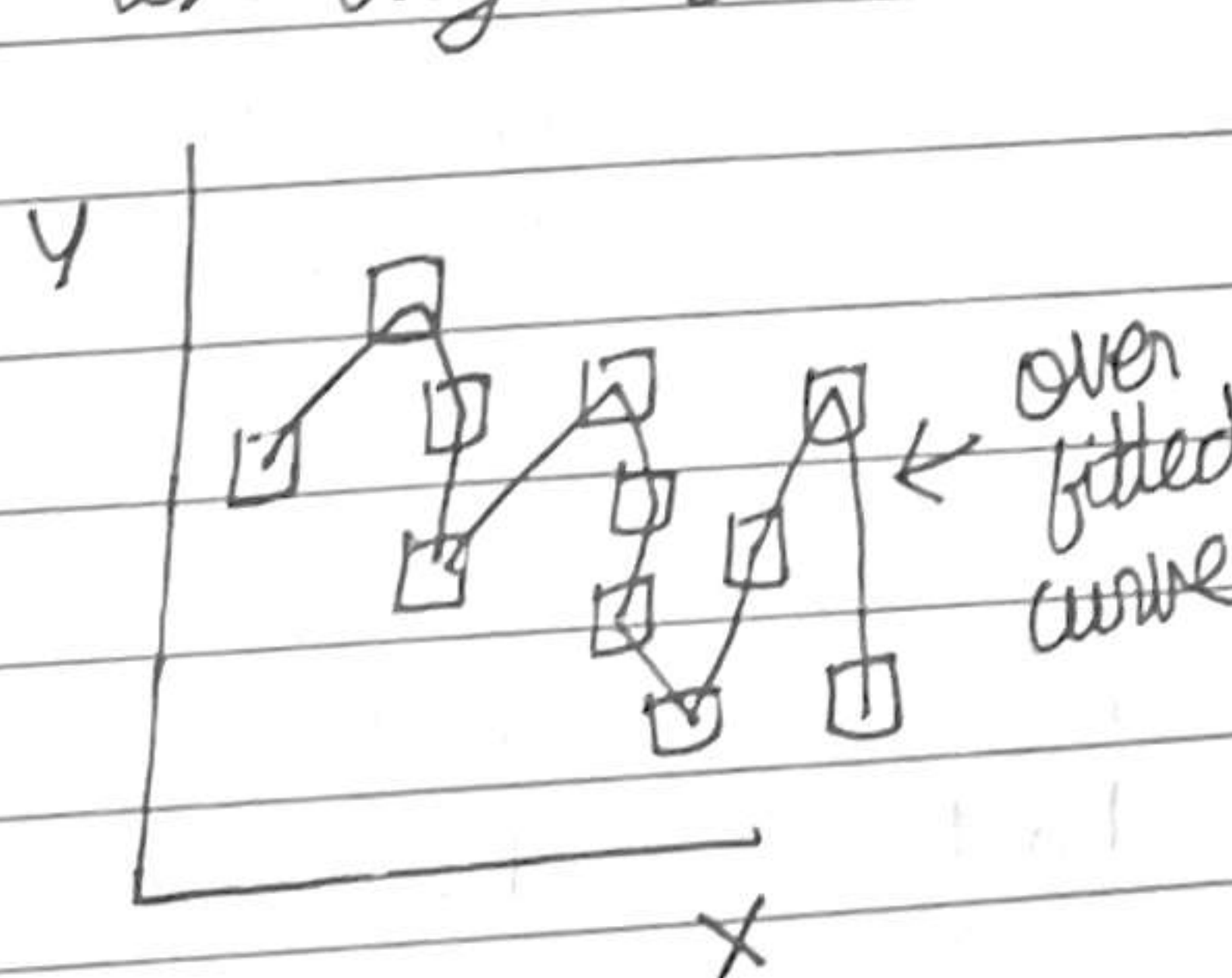
So, majority neighbors are pointing towards 'Normal'

Hence, class of (57, 170) should be 'Normal'

57	170	Normal
----	-----	--------

Overfitting vs Underfitting:

Aspect	Overfitting	Underfitting
1) Definition	Model learns both data patterns & noise (too complex)	Model fails to learn the data patterns (too simple)
2) Training Accuracy	Very high	Low
3) Testing Accuracy	Low (poor generalization)	Low
4) Model Complexity	High	Low
5) Cause	Too many parameters or too much training	Too few parameters or too little training
6) eg	Memorizing the training data	Linear model for non linear data
7) Solution	Simplify model, use regularization	Increase model complexity or add more features



Working of KNN Algorithm-

- 1) For implementing any algorithm, we need dataset. So during the first step of KNN, we must load the training as well as test data.
- 2) Next, we need to choose the value of K i.e. the nearest data points. K can be any integer.
- 3) For each point in the test data, do the following -
 - (i) Calculate the distance between test data & each row of training data with the help of any of the method namely: Euclidean, Manhattan or Hamming distance. Euclidean is most common.
 - (ii) Now, based on the distance value, sort them in ascending order.
 - (iii) Next, it will choose the top K rows from the sorted array.
 - (iv) Now, it will assign a class to the test point based on most frequent class of these rows.
- 4) End

Pros & Cons of KNN-

Pros of KNN-

- (i) Simple & easy to interpret
- (ii) Does not make any assumption so it can be implemented in non linear tasks
- (iii) Works well on classification with multiple classes
- (iv) Works well on both classification & regression tasks

Cons of KNN-

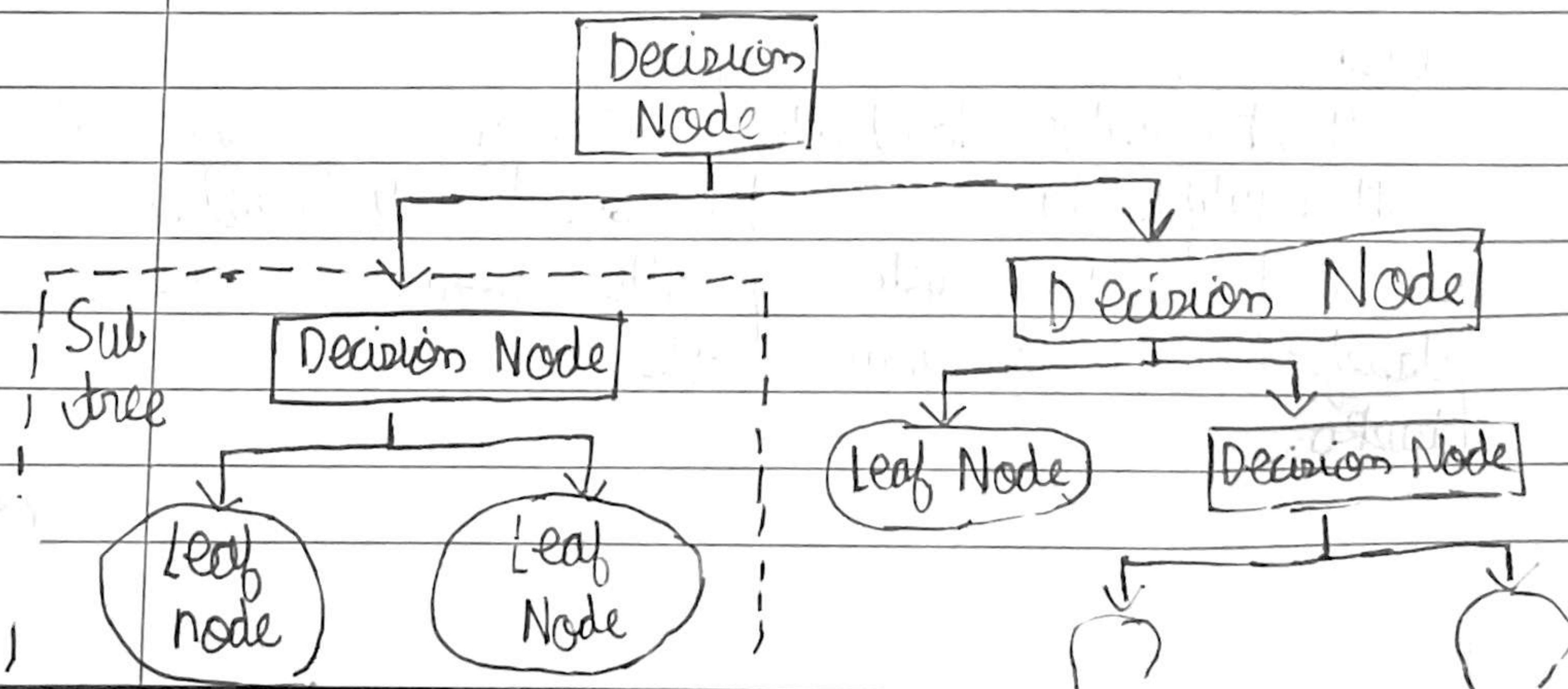
- (i) Becomes very slow as the number of data points increased because the model needs to store all data points
- (ii) Not memory efficient
- (iii) Sensitive to outliers. Outliers may also have a vote

Applications of KNN -

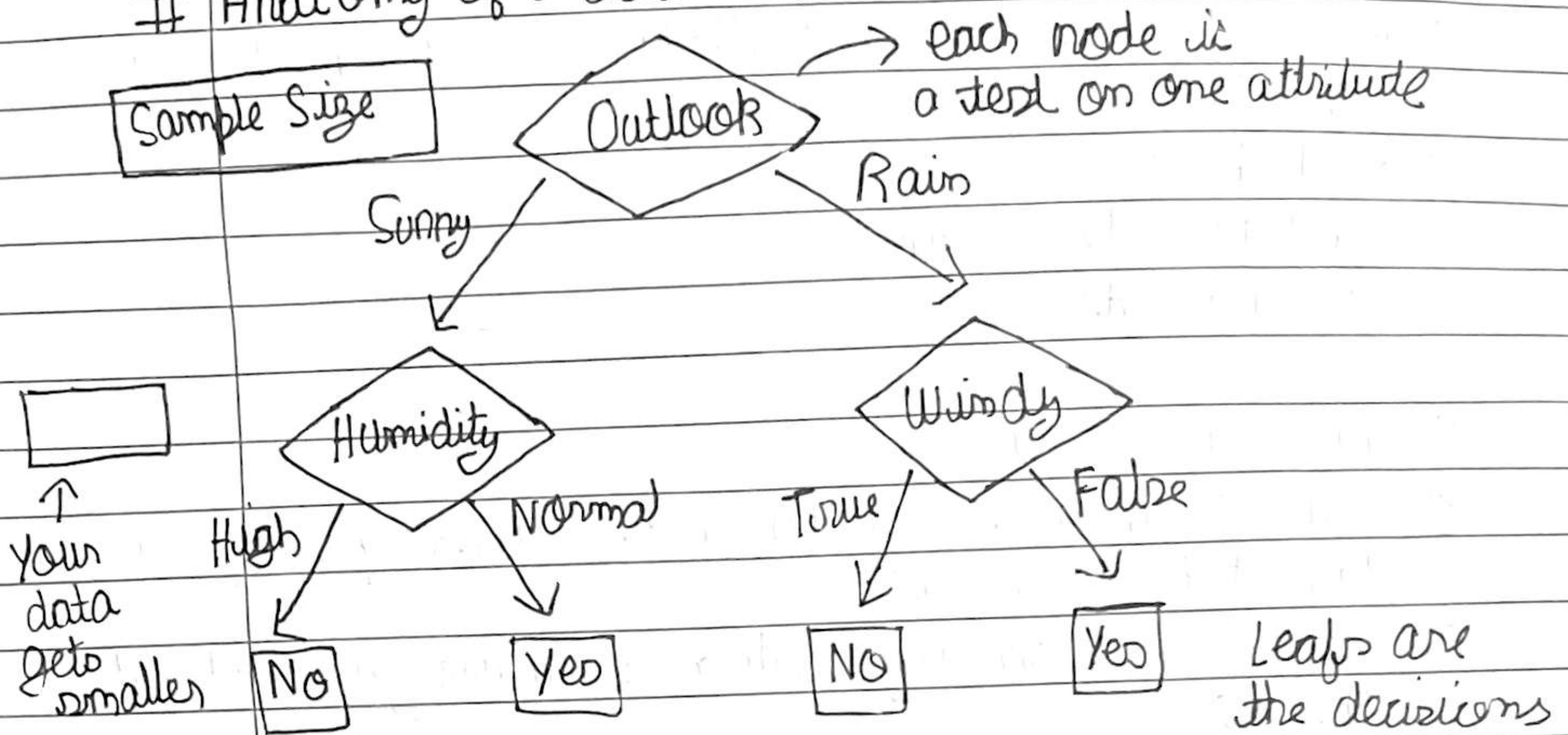
- 1) Credit Score
- 2) Approval of Loan
- 3) Data preprocessing (Imputation)
- 4) Prediction of stock prices
- 5) Recommendation Systems
- 6) Computer Vision

Decision Trees -

- 1) A supervised ~~training~~ learning technique used for ~~for~~ both classification and regression tasks
- 2) Mostly preferred in classification problems
- 3) Tree structured ~~node~~ model:
 - (a) Internal nodes = represent dataset features
 - (b) Branches = represent decision rules
 - (c) Leaf nodes = represent outcomes
- 4) Tree starts from a root node & expands into branches to form a tree like structure
- 5) Each node asks a question, and based on the Yes/No answer, it splits further into subtrees
- 6) Non linear classifier
- 7) Easy to use
- 8) Easy to interpret
- 9) Susceptible to overfitting but can be avoided



Anatomy of a decision tree -



10) Decision trees represent a disjunction of conjunctions of constraints on the attribute values of instances.

Eg.

(Outlook == Overcast)

OR

((Outlook == rain) and (not Windy == false))

11) In order to ~~use~~ build a tree, we use the CART algorithm, which stands for Classification & Regression Tree algorithm.

Decision tree algorithm -

- 1) Start with root node (S) containing the complete dataset
- 2) Identify the best attribute using an Attribute Selection Measure (ASM) such as Information Gain
- 3) Split the dataset (S) into subsets based on possible values of the selected attribute
- 4) Create a decision node for the best attribute
- 5) Repeat the process recursively for each subset until further splitting is not possible.

Attribute Selection Measures -

- 1) • Used To select the best attribute for the nodes of the tree
- 2) • 2 techniques
 - (a) Information Gain
 - (b) Gini Index

Information Gain -

- 1) Measures the reduction in entropy after splitting a dataset by an attribute
- 2) Shows how much information a feature gives about the target class
- 3) Used to decide where to split the node in a decision tree
- 4) The attribute with the highest information gain is chosen first

5) Formula:

$$\text{Information Gain} = \text{Entropy}(S) - [(\text{Weighted Avg}) * \text{Entropy}(\text{each feature})]$$

$$\text{Entropy}(S) = -\sum p_i \log_2(p_i) ; p_i = \text{probability of each class}$$

$$= -p_{\text{yes}} \log_2(p_{\text{yes}}) - p_{\text{no}} \log_2(p_{\text{no}}) ; \text{when 2 classes (Yes or no)}$$

Random Forest -

- 1) Used for classification & regression problems
- 2) Ensemble learning technique combining multiple decision trees
- 3) Each tree gives a prediction; majority vote determines the final output
- 4) More trees $\rightarrow$ higher accuracy & reduced overfitting
- 5) Assumption: Predictions from individual trees should have low correlation

Working of Random Forest algorithm -

Works in 2 phases:

Phase 1: Create the forest by combining N decision trees

Phase 2: Make predictions using each tree & take majority vote

Steps:

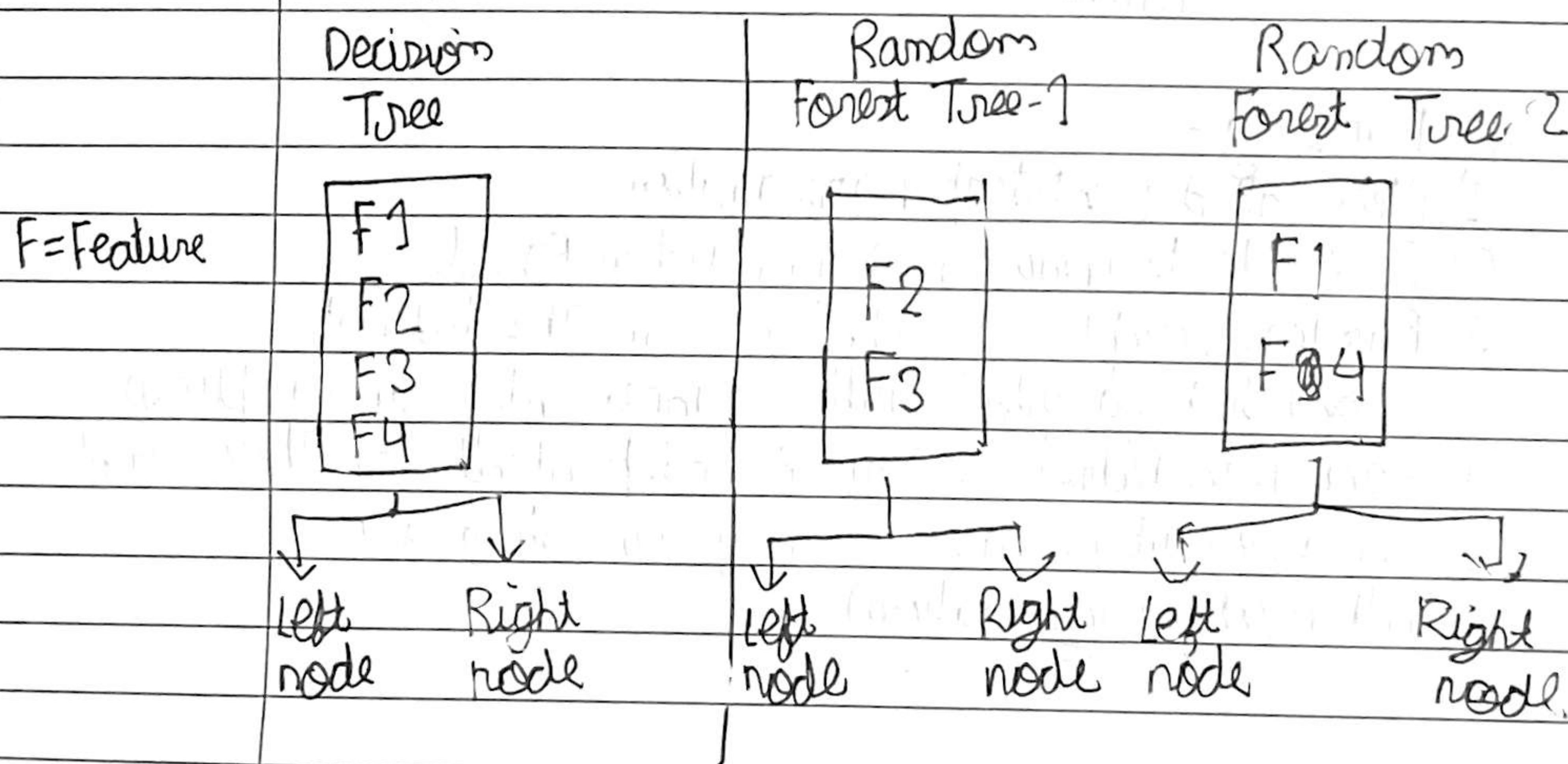
- 1) Select random K data points from the training set
- 2) Build decision trees using these subsets
- 3) ~~Divide~~ Decide N , the no. of trees to build
- 4) Repeat steps 1 & 2, until all trees are built
- 5) For new data, predict with each tree and assign the class with majority votes

eg - Dataset is given to the Random Forest Classifier

- Dataset is divided into subsets for each decision tree

- During training each tree makes predictions

- For new data, majority vote across trees determines the final prediction



- 6) Ensemble simply means combining multiple models
 7) Ensemble uses 2 types of methods

(a) Bagging:

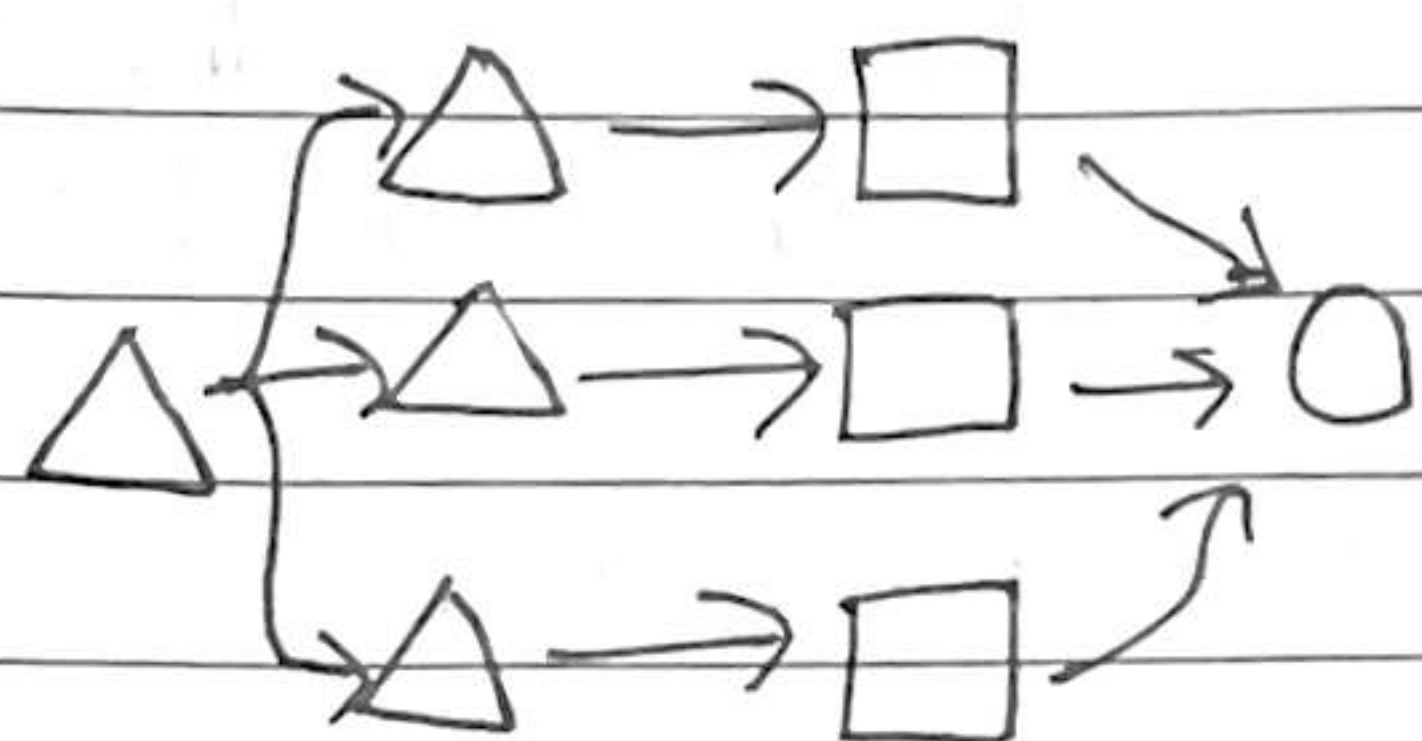
Creates a different training subset from sample training data with replacement & final output is based on majority vote.

Eg - Random Forest

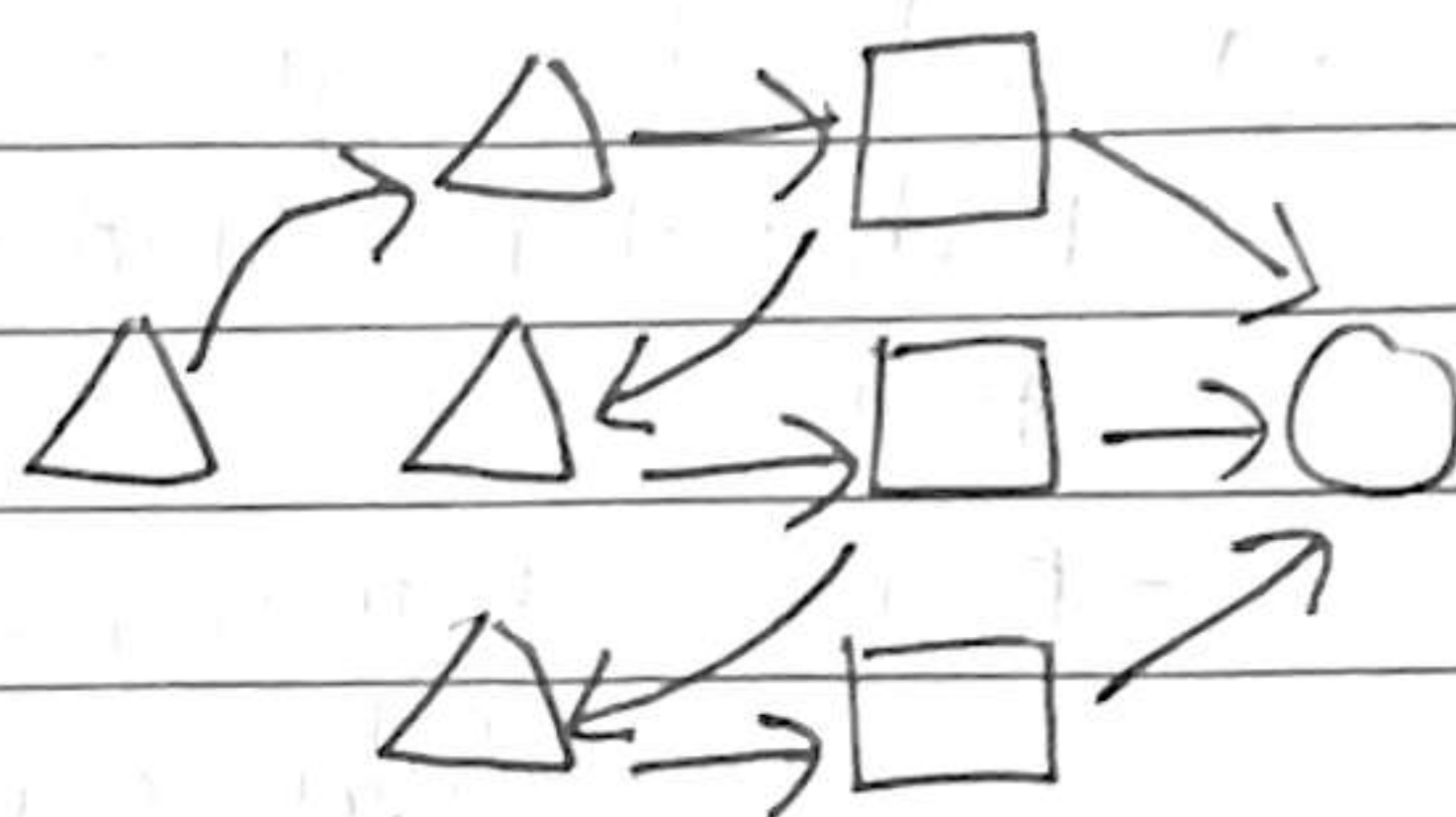
(b) Boosting:

Combines weak learners into strong learners by creating sequential models such that final model has the highest accuracy.

Eg - ADA Boost, XG Boost



Bagging
Parallel



Boosting
Sequential

Bagging -

- 1) Also called Bootstrap Aggregation
- 2) Ensemble technique used by Random Forest
- 3) Random samples are chosen from the dataset (bootstrap samples) with replacement (new sampling)
- 4) Each model/tree is trained independently on these samples
- 5) Final output is based on majority voting across all models (aggregation)

Features of Random Forest -

- 1) Diversity
- 2) Immune to the curse of dimensionality
- 3) Parallelization
- 4) Stability

Decision Tree vs Random Forest

Decision Tree	Random Forest
1) Tree like model of decisions along with possible outcomes in a diagram	1) A classification algorithm consisting of many decision trees
2) Scope of overfitting caused due to presence of too variance	2) Avoids & prevents overfitting
3) Results are not accurate	3) Accurate & precise results
4) Requires low computation	4) Consumes more computation
5) Low time	5) Time consuming
6) Easy to visualize	6) Complex to visualize

Hyperparameters -

- 1) Used in random forests to either enhance the performance of model or to make the model
- 2) Types -

(a) $n_estimators$ = no. of trees the algo builds before averaging the predictions

(b) $max_features$ = max no. of features algo considers before splitting a node

(c) min_sample_leaf = determines the min no. of leaves required to split an internal node

Support Vector Machine (SVM)

- 1) Supervised learning algorithm used for classification & sometimes regression
- 2) Goal is to find the best decision boundary (hyper plane) that separates data into classes
- 3) It uses extreme data points (called support vectors) to form this hyperplane
- 4) The hyperplane helps classify new data points correctly

5) Types:

(a) Linear SVM:

Used for linearly separable data - data can be divided by a single straight line

(b) Non Linear SVM:

Used for non linearly separable data - data that cannot be divided by a straight line

Linear vs Non Linear SVM

Linear SVM

- 1) It can be easily separated with a linear line
- 2) Data is classified with the help of hyperplane
- 3) Data can be easily classified by drawing a straight line

Non Linear SVM

- 1) It cannot be easily separated with a linear line
- 2) We use kernels to make non separable data into separable data
- 3) We map data into high dimensional space to classify

Advantages v/s Disadvantages of SVM

Advantages	Disadvantages
1) Good for smaller datasets	1) Not suitable for large datasets as the training time can be too too much
2) Accurate results	2) Not so effective on a dataset with overlapping classes
3) Useful for linear & non linearly separable data	3) Picking the right kernel can be computationally ext intensive
4) Effective in high dimensional space	

Applications of SVM -

- 1) Sentiment analysis
- 2) Spam detection
- 3) Image recognition
- 4) Handwritten digit recognition

Performance metrics for Regression -

- 1) Mean Absolute Error (MAE)
- 2) Mean Squared Error (MSE)
- 3) Root Mean Squared Error (RMSE)
- 4) R-Squared (R^2)

MAE formula: Formulae:

$$MAE = \frac{1}{n} \sum |Y - \hat{Y}|$$

Y = actual output
 $\hat{Y}$ = predicted output

$$MSE = \frac{1}{n} \sum (Y - \hat{Y})^2$$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum (Y - \hat{Y})^2}$$

R-Squared.

- 1) Also called coefficient of determination
- 2) Measures how ~~the~~ well a model fits the data
- 3) Shows how close the predicted values are to the actual value
- 4) Value ranges from 0 to 1 -
0: poor fit
1: perfect fit

$$R^2 = 1 - \frac{RSS}{TSS}$$

$$\text{Adjusted } R^2 = 1 - \left(\frac{(1-R^2)(n-1)}{n-k-1} \right)$$

$$\begin{aligned} RSS &= \text{Residual} \\ &\quad \text{Sum of Squares} \\ &= \sum (y - \hat{y})^2 \end{aligned}$$

$$\begin{aligned} TSS &= \text{Total Sum of} \\ &\quad \text{Squares} \\ &= \sum (y - \bar{y})^2 \end{aligned}$$

Evaluation Metrics (Pros & Cons)

1) MAE:

~~Pros~~ = Uses absolute errors, ignores directions; less
~~Cons~~ = ~~less~~ biased to large errors

2) MSE:

Uses square errors, considers direction; highly biased to large errors

3) RMSE:

Square root of MSE; penalizes large errors & better reflects overall performance

Performance Metrics for Classification:

- 1) Confusion matrix
- 2) Accuracy
- 3) Precision
- 4) Recall
- 5) F1 score

Confusion Matrix:

		Actual	
		1	0
Predicted	1	TP	FP
	0	FN	TN

→ Bad in ~~cancer~~ spam detection

↓ Bad in cancer detection

1) Accuracy -

No. of correct predictions made by the model
~~over all~~

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

Use when target variable classes in the data are nearly balanced

2) Precision -

$$\text{Precision} = \frac{TP}{TP + FP} \quad (\text{First row})$$

3) Recall or sensitivity

$$\text{Recall} = \frac{TP}{TP + FN} \quad (\text{First column})$$

4) F1 Score

$$F1 = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$